Mobilen Dergi 1

—

[2023-05-15 Mon]

İçindekiler

1 iOS SwiftUI Swipe Actions **2**

1.1 Swipe Actions nedir? 2

1.2 SwipeActions Kullanımı 3

1.3 Kaynak Kod Okuma: `PreferenceKey` kullanımı 3

1.4 Referanslar 4

2 Netfox ile iOS uygulamalarda network izlemek **5**

2.1 Netfox Nedir? 6

2.2 Netfox Nasıl Kullanılır? 6

2.3 Netfox'un Projeye Eklenmesi 6

2.4 Netfox Özetle 7

2.5 Referanslar 7

3 XCode Breakpoints **8**

1 iOS SwiftUI Swipe Actions

Suat Karakuşoğlu tarafından yazıldı.

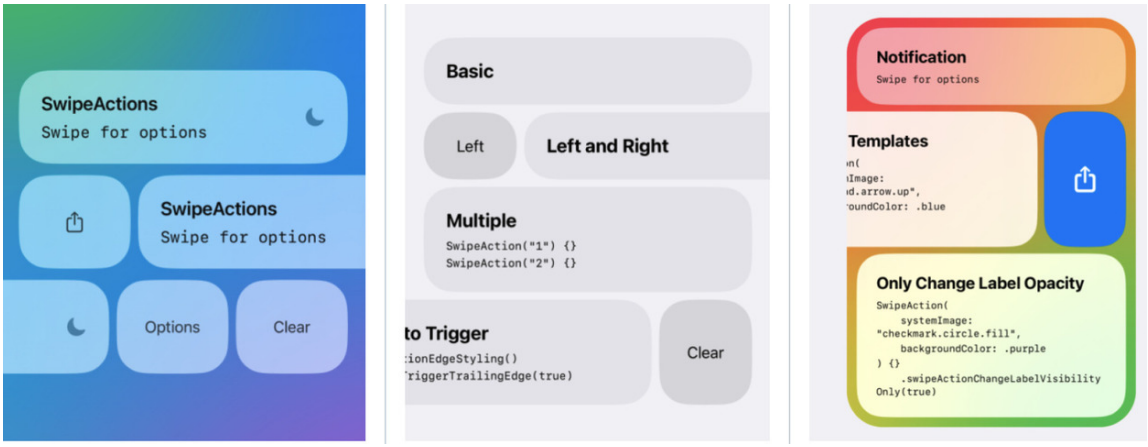

Figure 1: Örnek Swipe Actions

1.1 Swipe Actions nedir?

Merhabalar, herhangi bir ui elemanının üzerinden `swipe` ile çıkan aksiyonlar iOS kullanıcı deneyiminde önemli rol oynuyor.

Kullanıcı deneyimi açısından view'ın altında gizli, sağından veya solundan çıkan bu aksiyonlar görünmüyor olsa bile özellikle iOS kullanıcıları bu davranışı listelerde arayabiliyor.

Davranış olarak sağ ve sol demek yerine burada tercih edilen jargon `leading` ve `trailing`, bunun başlıca sebebi dillerin sağdan sola veya soldan sağa olarak 2 farklı şekilde yazılabilmesi. Soldan sağa yazılan dillerde sol-leading, sağ trailing iken, arapça-farsça gibi dillerde sol-trailing, sağ ise leading olmakta.

Swipe aksiyonları yazabilmek için yardımcı araçlar mevcut.

`Native` olarak SwiftUI tarafından iOS 15 ile SwiftUI List'elerine eklendi. iOS 15 SwiftUI SwipeActions adresinden native yöntem incelenebilir.

Ancak iOS-14 destekliyorsanız veya herhangi bir SwiftUI view'e swipe aksiyonları eklemek isterseniz bunun için güzel bir SwiftUI component'i açık kaynak olarak geliştirilip github'a `spm` destekli olarak koyulmuş.

Bu yazımızda Açık Kaynak SwipeActions Reposu'nu inceleyeceğiz.

1.2 SwipeActions Kullanımı

```
import SwiftUI
import SwipeActions

struct ContentView: View {
    var body: some View {
        SwipeView {
            Text("Hello")
              .frame(maxWidth: .infinity)
              .padding(.vertical, 32)
              .background(Color.blue.opacity(0.1))
              .cornerRadius(32)
        } trailingActions: { _ in
            SwipeAction("World") {
                print("Tapped!")
            }
        }
          .padding()
    }
}
```

Listing 1: Örnek bir swipe aksiyon eklemesi

1.3 Kaynak Kod Okuma: `PreferenceKey` kullanımı

Veri iletişimi view componentleri arasında Environment Objeler uzerinden `Parent` → `Child` view yönünde olabiliyor. Veya data binding ile `@Binding` çift taraflı `reactive` şekilde sağlanabiliyor.

PreferenceKey ile olan bu veri iletişiminde ise diğer bir ihtiyaç olan ise `child` → `parent` arasında veri geçişi yapabiliyoruz.

```
struct AllowSwipeToTriggerKey: PreferenceKey {
    static var defaultValue: Bool? = nil
    static func reduce(value: inout Bool?, nextValue: () -> Bool?)
    { value = nextValue() }
}
```

Listing 2: Preference Key tanımlama

Preference key tanımlarken ihtiyaç duyulan `PreferenceKey` protocol'unu `conform` etmek. Bunun için bir varsayılan değer `defaultValue` gerekiyor. Ayrıca ikinci olarak `reduce` fonksiyonu, bu fonksiyon dışarıdan sağlanan değerin preference olarak set edilmesine müdahale edilebilecek noktayı sağlıyor.

Kullanımı için 2 tane `modifier`'imiz mevcut: `preference` ve `onPreferenceChange`.

```
view.preference(key: AllowSwipeToTriggerKey.self, value: allowSwipeToTrigger)
```

Listing 3: Preference Key ile veriyi yukarı aktarma

```
view.onPreferenceChange(AllowSwipeToTriggerKey.self) { allow in
    /// Unwrap the value first (if it's not the edge action, `allow` is `nil`).
    if let allow {
        swipeToTriggerLeadingEdge = allow
    }
}
```

Listing 4: Preference Key ile aşağıdan gelen veriyi okuma

Ayrıca apple sdk'inde de `navigationTitle` modifier'i ile `preference key` kullanılarak `child` üzerinden çağırılan bir method ile üst view'deki `title`'i değiştirebiliyor.

Özetle hiyerarşide yukarı veri taşıyabilmek için kullanılan bu yapının kullanıldığını görüyoruz. Daha detaylı anlamak için Magic of View Preferences in SwiftUI yazısına göz atabilirsiniz.

1.4 Referanslar

1. Apple PreferenceKey Dokümantasyonu
2. Magic of View Preferences in SwiftUI

2 Netfox ile iOS uygulamalarda network izlemek

Uğurcan Durak tarafından yazıldı.

Figure 2: Netfox'u İzlemek için tıkla

Merhabalar, bu yazıda mobil cihazda network isteklerini detaylı şekilde görüntülemek için kullanılan faydalı olduğunu düşündüğüm **Netfox** kütüphanesinden bahsedicem.

Çok detaya girmeden genel hatlarıyla nedir? nasıl kullanılır? gibi başlıklarla değineceğim. Hadi başlayalım :)

2.1 Netfox Nedir?

Netfox kütüphanesi, tüm network işlemlerini mobil cihaz üzerinden detaylı bir şekilde görüntüleyebileceğimiz `open source` bir kütüphanedir.

Netfox'u uygulama içerisinde istediğimiz bir anda başlatıp anlık olarak uygulamamızın yapmış olduğu tüm istekleri listeleyebilir ve detaylı bir şekilde `body`, `header`, `response` gibi tüm network etkileşimlerini inceleyebiliriz.

Bu tarz kütüphanelerin en güzel yanı, mobil cihazımız bilgisayara bağlı olmadan da istediğimiz bir anda network log'larına bakabiliyor olmamızdır. Bu özellik testçiler için oldukça faydalıdır çünkü herhangi bir hata anında doğrudan developer'a bildirmek yerine log'ları inceleyip hata hakkında fikir sahibi olabilirler.

2.2 Netfox Nasıl Kullanılır?

Netflox'ın kullanımı ve projeye entegre edilmesi oldukça basittir.

Varsayılan davranışı **shake** olarak belirlenmiş olsada custom bir şekilde istediğimiz bir butona ya da bir view event'ine atayarak Netfox'ı başlatabiliriz. Öncelikle projeye nasıl eklenir ona bakalım.

2.3 Netfox'un Projeye Eklenmesi

1. Netflox kütüphanesini **SPM**, **CocoaPods** ve **Carthage** olmak üzere üç ayrı yöntem ile projemize ekleyebiliriz. Bu yöntemlerden herhangi biriyle eklediğinizi varsayarak bir sonraki adıma geçiyorum.
2. Kütüphaneyi projeye ekledik şimdi ise initiliaze edeceğiz. Bunun için AppDelegate class'ına gidip aşağıdaki gibi gerekli kodları ekliyoruz.

```
#if DEBUG
import netfox
#endif
```

Listing 5: Netfox Import

Burada **#if DEBUG** ve **#endif** kodlarını ekleyerek kodun sadece debug moddayken import olmasını sağlıyoruz. Daha sonra uygulama ilk başlatıldığında hazır hale gelmesini istediğim için aşağıdaki gibi başlatıcı fonksiyonun içerisine kodlarımızı ekliyoruz.

Burada **start()** methodunu çağırarak Netfox'ı initiliaze ediyoruz ardından **setGesture()** ile Netfox'ın nasıl açılacağını belirliyoruz.

```
func application(
  _ application: UIApplication,
  didFinishLaunchingWithOptions launchOptions: [UIApplication.LaunchOptionsKey: Any]?
) -> Bool {
    #if DEBUG
    NFX.sharedInstance().start()
    NFX.sharedInstance().setGesture(.shake)
    #endif
}
```

Listing 6: Netfox başlatma

Varsayılan davranışı shake olduğundan özellikle belirtmenize gerek yok örnek olması açısından ekledim. Siz dilerseniz **custom** parametresini geçip daha sonra dilediğiniz yerde **show()** methodu ile açılmasını sağlayabilirsiniz.

2.4 Netfox Özetle

Evet hepsi bu kadar :) Şimdi dilediğiniz gibi cihazı sallayarak ya da atamış olduğunuz bir buton ile Netfox'ı başlatabilir ve log'ları inceleyebilirsiniz.

Daha detaylı incelemek isteyenler için aşağıya link bırakıyorum. Umarım faydalı olur zaman ayırdığınız için teşekkürler.

2.5 Referanslar

1. Netfox Github Kütüphanesi
2. Örnek Kullanım Vidyosu

3 XCode Breakpoints

Alper Cem Öztürk tarafından yazıldı.